

argo-rdp.codeargo.net/web-rdp/#client/as0wnJfJNDU2MjQ0YzZjYyYwOABjAGpzb24?token=DA634F98C4DA7A4AC11136824A161C22BC460E45B3C7C7F80C9BC24825B85101

Applications: ML_Assignment1 - Mo... ML_Assignment1 - Thunar xfce Terminal

ML_Assignment1

localhost:8888/notebooks/ML_Assignment1.ipynb?

jupyter ML_Assignment1 Last checkpoint: 26 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

Python [conda env:base]

ML Assignment 1 - Bike Sharing Demand Prediction

Predicting hourly bike rentals using weather, time, and seasonal data. Metric used: RMSLE.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
import warnings
warnings.filterwarnings('ignore')

# RMSLE function as given in the assignment
def rmsle(y_true, y_pred):
    # Clip predictions to avoid log of negative values
    y_pred = np.clip(y_pred, 0, None)
    return np.sqrt(np.mean((np.log(y_pred) - np.log(y_true))**2))
```

```
[2]: # Load datasets
train = pd.read_csv('bike_train.csv')
test = pd.read_csv('bike_test.csv')

print("Train shape:", train.shape)
print("Test shape:", test.shape)
train.head()
```

Train shape: (18458, 12)
Test shape: (2613, 9)

```
[2]:
```

	datetime	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	casual	registered	count
0	2012-07-15 7:00:00	3	0	0	1	28.70000	33.335000	79	6.003200	17	30	47
1	2012-08-14 15:00:00	3	0	1	1	33.62000	37.880000	46	15.001300	84	199	283
2	2011-02-06 6:00:00	1	0	0	1	10.66000	12.880000	60	15.001300	0	1	1
3	2012-05-06 17:00:02	2	0	0	2	26.42506	30.566166	61	9.512288	198	330	531
4	2012-01-09 2:00:00	1	0	1	1	9.84000	12.120000	56	8.998100	2	3	5

Exploratory Data Analysis (EDA)

argo-rdp.codeargo.net/web-rdp/#/client/as0wNfJfNDU2MjQ0YzYzYwOABjAGpzb247token=DA634F98C4DA7A4AC1136824A161C22BC460E45B3C7C7F80C98C24825B85101

ML_Assignment1 ML_Assignment1 Thunar xfce Terminal

localhost:8888/notebooks/ML_Assignment1.ipynb?

jupyter ML_Assignment1 Last Checkpoint: 27 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python [conda env:base]

Q1. Examine dataset size, missing values, and feature types.

```
[3]: # Basic info about the dataset
print("Training set shape:", train.shape)
print("Test set shape:", test.shape)

print("\nFeature types:")
print(train.dtypes)

print("\nMissing values in train:")
print(train.isnull().sum())

print("\nMissing values in test:")
print(test.isnull().sum())

print("\nDescriptive statistics:")
train.describe()
```

Training set shape: (10450, 12)
Test set shape: (2613, 9)

Feature types:

datetime	object
season	int64
holiday	int64
workingday	int64
weather	int64
temp	float64
atemp	float64
humidity	int64
windspeed	float64
casual	int64
registered	int64
count	int64
dtype:	object

Missing values in train:

datetime	0
season	0
holiday	0
workingday	0
weather	0
temp	0
atemp	0
humidity	0
windspeed	0
casual	0
registered	0
count	0
dtype:	int64

Missing values in test:

argo-rdp.codeargo.net/web-rdp/#/client/s0wnJfJNDU2MjQ0YzZjYyYwOABjAGpzb24?token=DA634F98C4DA7A4AC1136824A161C22BC460E45B3C7C7F80C9BC24825B85101

ML_Assignment1

localhost:8888/notebooks/ML_Assignment1.ipynb?

Jupyter ML_Assignment1 Last Checkpoint: 27 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python [conda env:base] * ○ ○ ○

holiday and workingday are binary (0/1), and temp, atemp, humidity, windspeed are continuous numeric features.

The target is 'count' (total bikes rented per hour). There's also 'casual' and 'registered' which add up to 'count', but these won't be available in the test set so we can't use them as features.

Q2. Visualize relationships between key features and the target variable (count).

```
[4]: # Parse datetime for visualization
train['datetime'] = pd.to_datetime(train['datetime'])
train['hour'] = train['datetime'].dt.hour
train['month'] = train['datetime'].dt.month
train['dayofweek'] = train['datetime'].dt.dayofweek

fig, axes = plt.subplots(2, 3, figsize=(16, 10))

# Count vs Hour
train.groupby('hour')['count'].mean().plot(ax=axes[0, 0], markers='o')
axes[0, 0].set_title('Average Count by Hour')
axes[0, 0].set_xlabel('Hour')
axes[0, 0].set_ylabel('Average Count')

# Count vs Season
train.groupby('season')['count'].mean().plot(kind='bar', ax=axes[0, 1], color='skyblue')
axes[0, 1].set_title('Average Count by Season')
axes[0, 1].set_xlabel('Season')
axes[0, 1].set_ylabel('Average Count')

# Count vs Temperature
axes[0, 2].scatter(train['temp'], train['count'], alpha=0.1, s=5)
axes[0, 2].set_title('Count vs Temperature')
axes[0, 2].set_xlabel('Temperature')
axes[0, 2].set_ylabel('Count')

# Count vs Weather
train.groupby('weather')['count'].mean().plot(kind='bar', ax=axes[1, 0], color='salmon')
axes[1, 0].set_title('Average Count by Weather')
axes[1, 0].set_xlabel('Weather')
axes[1, 0].set_ylabel('Average Count')

# Count vs Humidity
axes[1, 1].scatter(train['humidity'], train['count'], alpha=0.1, s=5)
axes[1, 1].set_title('Count vs Humidity')
axes[1, 1].set_xlabel('Humidity')
axes[1, 1].set_ylabel('Count')

# Count vs Working Day (by hour)
train[train['workingday']==1].groupby('hour')['count'].mean().plot(ax=axes[1, 2], label='Working Day', markers='o')
train[train['workingday']==0].groupby('hour')['count'].mean().plot(ax=axes[1, 2], label='Non-Working Day', markers='s')
axes[1, 2].set_title('Hourly Count: Working vs Non-Working Day')
axes[1, 2].legend()
```

argo-rdp.codeargo.net/web-rdp/#/client/s0wnJfJNDU2MjQ0YzZjYyYwOABjAGpzb24?token=DA634F98C4DA7A4AC1136824A161C22BC460E45B3C7C7F80C9BC24825B85101

ML_Assignment1

localhost:8888/notebooks/ML_Assignment1.ipynb?

Jupyter ML_Assignment1 Last Checkpoint: 27 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python [conda env:base] * ○ ○ ○

```
[5]: # Correlation heatmap
plt.figure(figsize=(10, 8))
# Use only numerical features available in test set plus derived ones
corr_cols = ['season', 'holiday', 'workingday', 'weather', 'temp', 'atemp', 'humidity', 'windspeed', 'hour', 'month', 'count']
sns.heatmap(train[corr_cols].corr(), annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```

Correlation Heatmap

	season	holiday	workingday	weather	temp	atemp	humidity	windspeed	hour	month	count
season	1.00	0.03	-0.00	0.01	0.26	0.27	0.18	-0.15	0.00	0.97	0.16
holiday	0.03	1.00	-0.25	-0.01	0.00	-0.01	0.00	0.00	-0.01	-0.00	-0.01
workingday	-0.00	-0.25	1.00	0.02	0.04	0.03	-0.02	0.02	-0.00	-0.00	0.02
weather	0.01	-0.01	0.02	1.00	-0.05	-0.05	0.40	0.01	-0.02	0.01	-0.12
temp	0.26	0.00	0.04	-0.05	1.00	0.98	-0.07	-0.01	0.15	0.26	0.40
atemp	0.27	-0.01	0.03	-0.05	0.98	1.00	-0.04	-0.05	0.14	0.27	0.39
humidity	0.18	-0.00	-0.02	0.40	-0.07	-0.04	1.00	-0.32	-0.27	0.20	-0.32
windspeed	-0.15	0.00	0.02	0.01	-0.01	-0.05	-0.32	1.00	0.14	-0.15	0.11
hour	0.00	-0.01	-0.00	-0.02	0.15	0.14	-0.27	0.14	1.00	0.00	0.40

argo-rdp.codeargo.net/web-rdp/#/client/aS0wNfJNDU2MjQ0YzYzYyYwOABjAGpzb247token=DA634F98C4DA7A4AC1136824A161C228C460E45B3C7C7F80C9BC248258B5101

ML_Assignment1 — Mo... ML_Assignment1 - Thunar Xfce Terminal

localhost:8888/notebooks/ML_Assignment1.ipynb?

jupyter ML_Assignment1 Last Checkpoint: 28 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python [conda env:base]

Applied **sinus encoding** for hour and month — this makes the model treat hour 22 and hour 0 as close (since the wrap-around; same idea for December and January).

- Created a **rush_hour** flag (1 if it's 7-9 AM or 4-7 PM on a working day) to directly capture the commute pattern.
- Added an **is_weekend** binary feature.
- One-hot encoded** season and weather since they're categorical — treating them as raw numbers would imply season 4 is "four times" season 1 which doesn't make sense.

These help the linear model pick up patterns that are otherwise invisible to it (like the nonlinear hourly effect).

Regression Models

Q5. Split data and build a simple Linear Regression model.

```
[9]: # Split into train and validation (80-20 split)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Training set size: {X_train.shape[0]}")
print(f"Validation set size: {X_val.shape[0]}")

# Simple Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Predictions
y_train_pred_lr = lr_model.predict(X_train)
y_val_pred_lr = lr_model.predict(X_val)

# Calculate RMSE
train_rmse_lr = rmse(y_train, y_train_pred_lr)
val_rmse_lr = rmse(y_val, y_val_pred_lr)

print(f"\nLinear Regression Results:")
print(f"  Train RMSE: {train_rmse_lr:.4f}")
print(f"  Validation RMSE: {val_rmse_lr:.4f}")

Training set size: 8360
Validation set size: 2090

Linear Regression Results:
  Train RMSE: 1.1643
  Validation RMSE: 1.1238
```

Q6. Polynomial features + Ridge and Lasso regression with tuning.

```
[10]: # Polynomial features (degree 2)
# Using a subset of key numeric features to avoid explosion of dimensions
poly_features = ['temp', 'humidity', 'windspeed', 'hour', 'month', 'hour_sin', 'hour_cos']
```

The screenshot shows a Jupyter Notebook titled "ML_Assignment1". The code defines a Lasso regression model with interactions and log target, and prints the best parameters. Below the code, the output displays the cross-validated RMSE values for different alpha values. A horizontal line separates the output from the next section.

```
# Best Lasso (log target + interactions) alpha: (best_lasso_log_alpha), Val RMSLE: (best_lasso_log_rmse:.4f)

Lasso on interactions + log target:
alpha=0.001: Val RMSLE=0.5306
alpha=0.005: Val RMSLE=0.5490
alpha=0.01: Val RMSLE=0.5602
alpha=0.05: Val RMSLE=0.6313
alpha=0.1: Val RMSLE=0.6935
alpha=1: Val RMSLE=1.4249
```

Best Lasso (log target + interactions) alpha: 0.001, Val RMSLE: 0.5306

Model Comparison and Interpretation

Q7. Summary table of all models.

```
[17]: # Collect all results
results = []

# 1. Simple Linear Regression
results.append({
    'Model': 'Linear Regression',
    'Features': 'Engineered features',
    'Target Transform': 'None',
    'Train RMSLE': rmsle(y_train, lr_model.predict(X_train)),
    'Val RMSLE': rmsle(y_val, lr_model.predict(X_val)),
    'Key Observation': 'Baseline model'
})

# 2. LR on Poly2 subset
results.append({
    'Model': 'Linear Regression + Poly2',
    'Features': 'Polynomial (subset)',
    'Target Transform': 'None',
    'Train RMSLE': rmsle(y_train, lr_poly2.predict(X_train_poly2)),
    'Val RMSLE': rmsle(y_val, lr_poly2.predict(X_val_poly2)),
    'Key Observation': 'Polynomial helps capture nonlinearity'
})

# 3. Ridge on Poly2 subset
results.append({
    'Model': f'Ridge (alpha={best_ridge_alpha}) + Poly2',
    'Features': 'Polynomial (subset, scaled)',
    'Target Transform': 'None',
    'Train RMSLE': rmsle(y_train, best_ridge_model.predict(X_train_poly2_scaled))
})
```



argo-rdp.codeargo.net/web-rdp/#client/a50wHfJfNDU2MjQ0YzZjYWYwOABJAQpzb247token=DA634F98C4DA7A4AC11136824A161C22BC460E4583C7C7F80C98C24825B85101

ML_Assignment1 ML_Assignment1 Thunar Xfce Terminal

localhost:8888/notebooks/ML_Assignment1.ipynb?

jupyter ML_Assignment1 Last Checkpoint: 29 minutes ago

File Edit View Run Kernel Settings Help

JupyterLab Python [conda env:base] Not Trusted

terms. That's basically what we did — the cyclical encoding and interaction with workingday let the model "see" these nonlinear, context-dependent time effects.

Final Model: Generate Predictions for Test Set

```
[20]: # Retrain the best model on FULL training data
# Best model: Ridge with interactions + log target

# Prepare full training data
X_full = X
y_full_log = np.log1p(y)

# Create interaction features for full training and test
poly2_final = PolynomialFeatures(degree=2, include_bias=False, interaction_only=True)
X_full_poly = poly2_final.fit_transform(X_full)
X_test_poly = poly2_final.transform(X_test_final)

print(f"Full training features shape: {X_full_poly.shape}")
print(f"Test features shape: {X_test_poly.shape}")

# Scale
scaler_final = StandardScaler()
X_full_poly_sc = scaler_final.fit_transform(X_full_poly)
X_test_poly_sc = scaler_final.transform(X_test_poly)

# Train final model with the best alpha found earlier
final_model = Ridge(alpha=best_log_alpha)
final_model.fit(X_full_poly_sc, y_full_log)

# Predict on test set
y_test_pred_log = final_model.predict(X_test_poly_sc)
y_test_pred = np.exp1p(y_test_pred_log)
y_test_pred = np.clip(y_test_pred, 0, None) # ensure non-negative predictions

print(f"\nPrediction statistics:")
print(f"  Min: {y_test_pred.min():.2f}")
print(f"  Max: {y_test_pred.max():.2f}")
print(f"  Mean: {y_test_pred.mean():.2f}")
print(f"  Any NaN: {np.isnan(y_test_pred).any()}")

Full training features shape: (10450, 253)
Test features shape: (2613, 253)

Prediction statistics:
Min: 1.56
Max: 877.86
Mean: 187.39
Any NaN: False
```

argo-rdp.codeargo.net/web-rdp/#/client/aS0wNfJNDU2MjQ0YzZjYTYwOABjAGpzbZ47token=DA634F98C4DA7A4AC11136824A161C22BC460E45B3C7C7F80C98C24825B85101

ML_Assignment1 x ML_Assignment1 - Thunai Xfce Terminal

localhost:8888/notebooks/ML_Assignment1.ipynb?

jupyter ML_Assignment1 Last Checkpoint: 29 minutes ago

File Edit View Run Kernel Settings Help

Not Trusted

JupyterLab Python [conda env:base] +

```
[21]:
```

	datetime	count_predicted
0	05-06-2012 5:00	14.847198
1	19-03-2011 19:00	179.427700
2	02-04-2012 6:00	26.588440
3	13-07-2012 20:00	362.505831
4	09-11-2011 19:00	313.389296
5	10-03-2012 0:00	112.924929
6	19-05-2011 13:00	98.731936
7	01-05-2011 8:00	67.321569
8	07-07-2012 8:00	129.896819
9	07-05-2011 16:00	454.891954

```
[22]:
```

```
# Save submission file
submission.to_csv('submission.csv', index=False)
print("submission.csv saved successfully!")
print(f"File has {len(submission)} rows and {len(submission.columns)} columns")

submission.csv saved successfully!
File has 2613 rows and 2 columns
```

```
[23]:
```

```
# Final verification - read back and check
verify = pd.read_csv('submission.csv')
print("Verification of submission.csv:")
print(f" Shape: {verify.shape}")
print(f" Columns: {list(verify.columns)}")
print(f" count_predicted dtype: {verify['count_predicted'].dtype}")
print(f" Any nulls: {verify.isnull().any().any()}")
print(f" count_predicted range: [{verify['count_predicted'].min():.2f}, {verify['count_predicted'].max():.2f}]")
print("\nAll checks passed!")

Verification of submission.csv:
Shape: (2613, 2)
Columns: ['datetime', 'count_predicted']
count_predicted dtype: float64
Any nulls: False
count_predicted range: [1.56, 877.86]

All checks passed!
```

So to wrap up — the best model is **Ridge Regression (alpha=1)** trained on interaction features with log1p(count) as the target, giving a validation RMSLE of **0.5228**. The log transform was the single biggest improvement, and interaction features + regularization pushed it further. The submission.csv is generated using this model retrained on the full training set.